REPUBLIC OF TURKEY
YILDIZ TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

DESIGN AND IMPLEMENTATION OF AN INSTITUTION-SPECIFIC LARGE LANGUAGE MODEL-BASED INTELLIGENT QUESTION ANSWERING SYSTEM

21011088 — CEYHUN KIRCA

SENIOR PROJECT

Advisor
Prof. Dr. Mine Elif Karslıgil

June, 2026

ACKNOWLEDGEMENTS

I would like to extend my sincerest thanks to everyone who contributed to the development of this project and who supported me throughout the process.

First and foremost, I would like to express my deepest gratitude to my esteemed advisor, Prof. Dr. Mine Elif Karslıgil, whose valuable knowledge and experience guided this project in the right direction at every stage.

I would like to extend my special thanks to Merve Zenginerler from the Department of Architecture and her valuable advisor, Prof. Dr. Zeynep Gül Ünal, for their invaluable expertise in their respective fields. Their vital contributions to the formulation of the specific set of architectural questions and the rigorous qualitative evaluation of the model's outputs have played a crucial role in validating the practical effectiveness of this system.

I am also grateful to Yıldız Technical University and the Department of Computer Engineering for their significant contributions to my academic development throughout my education.

Finally, I want to thank my family and friends for their moral and motivational support throughout this process.

CEYHUN KIRCA

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Approximate Nearest Neighbor
DPR	Dense Passage Retrieval
HNSW	Hierarchical Navigable Small World
LLM	Large Language Model
LPU	Language Processing Units
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval Augmented Generation Assessment
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine-Tuning

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

Design and Implementation of an Institution-Specific Large Language Model-Based Intelligent Question Answering System

CEYHUN KIRCA

Department of Computer Engineering
Senior Project

Advisor: Prof. Dr. Mine Elif Karslıgil

This project develops a question answering system for the fields of architecture and cultural heritage preservation using RAG architecture. The main objective of the study is to mitigate the hallucination problem frequently encountered when querying Large Language Models in highly technical and detailed fields and to ensure that the models produce consistent and reliable answers based solely on relevant documents. To this end, a massive dataset of approximately 130 GB, consisting of over 8,000 architectural and heritage preservation documents has been prepared and processed as a high throughput, dense vector database. The system is designed to rely entirely on relevant information sources for the generation process, rather than relying on the models memory. Finally, a classic, simple, and easy-to-follow chatbot interface was designed.

The project's methodology involves an advanced and multi stage architecture. First, document parsing, cleaning and vector generation were performed using the BAAI/bge-m3 model and the resulting data was stored in chromaDB for efficient semantic searching. The BAAI/bge-m3 model was chosen due to its robust structure, multi language support and successful results. To maximize information retrieval accuracy, a two-stage retrieval pipeline was established by integrating the BAAI/bge-reranker-v2-m3 Cross Encoder model. Dynamic score thresholds were also defined to pre-filter low quality text fragments. During the generation phase three different popular open-source language models (Llama-3.1-8B, Llama-3.3-70B and Qwen-32B) were integrated into the system via high speed LPU's and tested to

compare their performance. To improve the reasoning and output generation accuracy of the models, the generation process was further optimized using prompt strategies called thought chains.

The system's performance was carefully measured using an automated RAGAS evaluation framework and blind tests conducted by domain experts. During the experimental phase, various retrieval configurations, the number of chunks retrieved(k), prompt designs and threshold limits were tested and compared. The results demonstrated that the highest performance was achieved by combining a large initial semantic search (initial k=40) cross-encoder reordering phase (Final K=15) and prompt strategy called CoT. This configuration boosted critical evaluation metrics such as faithfulness, answer relevance, and context precision to over %91, surpassing all other test results and peaking with a Weighted RAG Score of 0.931.

In conclusion, this study has proven through testing that implementing a two-stage RAG architecture equipped with rigorous filtering and reasoning prompts largely eliminates model hallucinations and produces successful outputs. The findings demonstrate that even open-source and low parameter language models, when properly supported by advanced information retrieval tools, can generate highly accurate, reliable, and professional responses in areas requiring expertise and detail, without the need for computationally costly large parameter models.

Keywords: Large Language Models,Natural Language Processing,RAG,Semantic Search

ÖZET

Kurum-Özgü Büyük Dil Modeli Tabanlı Akıllı Soru-Cevap Sistemi Tasarımı ve Gerçekleştirilmesi

CEYHUN KIRCA

Bilgisayar Mühendisliği Bölümü
Bitirme Projesi

Danışman: Prof. Dr. Mine Elif Karslıgil

Bu projede, RAG mimarisi kullanılarak mimarlık ve kültürel mirasın korunması alanlarına yönelik bir soru cevap sistemi geliştirilmiştir. Çalışmanın temel amacı, Büyük Dil Modellerinin son derece teknik ve detaylı alanlarda sorgulandığında sıklıkla karşılaşılan halüsinasyon problemini hafifletmek ve modellerin yalnızca ilgili dokümanlara dayanarak tutarlı ve güvenilir cevaplar üretmesini sağlamaktır. Bu doğrultuda, yaklaşık 130 GB boyutunda, 8.000'den fazla mimari doküman ve miras koruma belgelerinden oluşan devasa bir veri seti hazırlanmış ve yüksek verimliliğe sahip yoğun bir vektör veritabanı olarak işlenmiştir. Sistem, modellerin hafızasına güvenmek yerine üretim sürecini tamamen ilgili bilgi kaynaklarına dayandıracak şekilde tasarlanmıştır. En sonunda da klasik,sade ve kolay takip edilebilir bir chatbot arayüzü tasarlanmıştır.

Projenin methodu, gelişmiş ve çok aşamalı bir mimariyi içermektedir. İlk olarak, dokümanların ayrıştırılması,temizlenmesi ve vektör üretimi BAAI/bge-m3 modeli kullanılarak gerçekleştirilmiş ve elde edilen veriler, verimli anlamsal arama için ChromaDB'de depolanmıştır. BAAI/bge-m3 modeli güçlü yapısı ve çok dil desteği barındırması ve başarılı sonuçları nedeniyle tercih edilmiştir. Bilgi getirme doğruluğunu maksimize etmek amacıyla, BAAI/bge-reranker-v2-m3 Cross Encoder modeli entegre edilerek 2 aşamalı retrieval verihattı kurulmuştur. Ayrıca düşük kaliteli metin parçalarını önceden filtrelemek için dinamik skor barajları tanımlanmıştır. Üretim aşamasında ise açık kaynaklı üç farklı popüler dil modeli (Llama-3.1-8B,

Llama-3.3-70B ve Qwen-32B), yüksek hızlı LPU'lar aracılığıyla sisteme entegre edilmiş ve test edilerek başarıları kıyaslanmıştır. Modellerin akıl yürütme ve çıktı üretme doğruluklarını artırmak için üretim süreci düşünce zinciri adı verilen prompt stratejileriyle daha da optimize edilmiştir.

Sistemin performansı, otomatik RAGAS değerlendirme çerçevesi ve alan uzmanları tarafından yapılan kör testler kullanılarak dikkatlice ölçülmüştür. Deneysel aşamada; çeşitli retrieval yapılandırmaları, getirilen chunk sayısı(k), prompt tasarımları ve eşik sınırları denenerek analiz edilmiş ve karşılaştırmalar yapılmıştır. Çıkan sonuçlar,en yüksek performansın; geniş çaplı bir ilk anlamsal arama Başlangıç k=40 ile çapraz kodlayıcı yeniden sıralama Final k=15 aşamasının ve CoT adı verilen prompt stratejisinin birleştirilmesiyle elde edildiğini kanıtlamıştır. Bu yapılandırma, bağlam gassasiyeti, sadakat ve cevap uygunluğu gibi kritik değerlendirme metriklerini %91'in üzerine taşımış ve ve diğer tüm test sonuçlarını geride bırakarak 0.931 Ağırlıklı RAG Skoru ile zirveye ulaşmıştır.

Sonuç olarak bu çalışma, katı anlamsal filtreleme ve bilişsel akıl yürütme promptları ile donatılmış 2 aşamalı bir RAG mimarisinin uygulanmasının, model halüsinasyonlarını büyük ölçüde ortadan kaldırdığını ve başarılı çıktılar ürettiğini kanıtlamıştır. Elde edilen bulgular; açık kaynaklı ve düşük parametreli dil modellerinin dahi, gelişmiş bilgi getiri mekanizmaları ile doğru şekilde desteklendiğinde, hesaplama açısından maliyetli model ince ayar işlemlerine gerek kalmadan, uzmanlık gerektiren alanlarda son derece doğru, güvenilir ve profesyonel yanıtlar üretebileceğini ortaya koymuştur.

Anahtar Kelimeler: Büyük Dil Modelleri,Doğal Dil İşleme,RAG,Anlamsal Arama

1
Introduction

Recent advancements in artificial intelligence, particularly in LLMs have significantly enhanced the ability of systems to understand and generate natural language. These advancements have enabled more diverse approaches to accessing information and question answering. However, applying such models directly to institution-specific and closed-source data presents several critical challenges, including data privacy concerns, lack of context-dependent response and risk of generating unreliable or misleading answers.

In many organizations, essential information is stored in unstructured formats such as internal documents, reports and manuals. Using traditional keyword-based search systems to efficiently access this information is often difficult, inefficient, has a low success rate. This systems frequently fail to capture semantic relationships between user queries and document content, leading to inefficient and sometimes irrelevant and incorrect results and guidance.

To overcome these limitations, there has been a growing need for intelligent systems that can understand both questions and documents and provide accurate, reliable and document-based answers. This project aims to adress this need by designing an institution-specific intelligent question answering system that leverages recent advances in language models and information retrieval techniques.

1.1 Purpose

The primary goal of this project is to design an organization specific, intelligent question answering system infrastructure and subsequent chatbot that utilizes LLMs. The system aims to analyze the documents within the system and provide accurate, source-referenced answers to user questions expressed in natural language.

The primary goal of proposed system is ensure that all generated responses are based on institutional data and supported by source references. This approach aims to

minimize missleading results and improve the system's reliability, consistency and performance. Furthermore, the project prioritize minimizing the time required for users to access critical information.

Another important goal is to differentiate the system from traditional searches by using semantic search that understands the user's intent and the true meaning of the documents. During the project's development phase, a reranking process was tested and proven successful, double checking results and completely eliminating irrelevant data to ensure the system finds the most accurate information. Furthermore, the project encourages step-by-step thinking by carefully designing instructions given to model, thus enabling it to assemble the obtained information in a natural and consistent manner. By combining this precise search method with a well-directed language model, the system aims to provide clear, reliable and accurate answers that standard search tools cannot offer.

1.2 Preliminary Review

This preliminary review focuses on current research in the fields of information retrieval, semantic search and LLM-based question and answer systems. Traditional information retrieval systems rely on approaches that are limited in capturing the meaning of user queries and documents. As a result, these systems often fail to provide accurate and relevant results in complex information retrieval scenarios and resort to fabrications and leading to misinformation.

Recent advances in semantic search and representation learning have led to embedding-based methods that model textual data in high-dimensional vector spaces. These approaches provide more effective matching between queries and documents based on context and capturing semantic relationships, rather than relying solely on keyword overlap.

The approach we used in this project RAG, has emerged as a powerful method that combines recall mechanisms with generative language models. RAG is defined as a tool that integrates parametric and non-parametric memory and allows language models to access external information sources during generation. This approach significantly improves the performance of question answering systems, especially in information intensive tasks.[1].

Subsequent studies have also shown that RAG-based systems improve accuracy and reduce hallucination by basing the generated responses on the acquired documents[1]. This is the main goal of our project and the main reason why chosen the RAG structure.

On the other hand, recent research highlights that RAG architectures specialize in contextual understanding and provide more reliable outputs compared to standalone language models.[2].

Various domain-specific applications of RAG have also been proposed in the literature. For example, RAG-based systems have been successfully applied in document-based question answer studies involving complex documents where advanced search and process techniques are required [1]. Similarly, studies focus on understanding based information access process highlight the importance of embedding models and vector based search methods in improving access success and system performance. [3].

Despite these advances, many current approaches are designed for open field applications and may not adequately address the requirements of organization-specific environments such as data privacy, data access and verifiability of responses. Additionally, challenges such as data retrieval quality and illusion reduction remain active areas of research.

Therefore, this project aims to build upon existing RAG-based approaches by developing a system that works with institution-specific data, reducing hallucinations, increasing accuracy and discovering new findings. The proposed system focuses on combining and testing semantic search, cross-encoder reordering, vector-based retrieval, cognitive orientation strategies and LLMs to generate accurate, contextually faithful and referencing responses in a close loop environment.

2
Literature Analysis

Recent advancements in AI and NLP have significantly enhanced the capabilities of question answering systems. In particular, the emergence of LLMs has enabled systems to produce highly consistent and successful responses. However, standalone LLMs are often limited by hallucinations, outdated information, or domain-specific general knowledge. To overcome these limitations, RAG architectures have become one of the most effective approaches in natural language processing tasks.[1].

RAG combines information retrieval techniques with neural text generation by retrieving relevant documents and conditioning the production process according to that context. This hybrid architecture improves consistency, contextual relevance and explainability compared to purely generative systems. Integrating retrieval mechanisms into production models significantly enhance performance[1].

In recent years, research interest in RAG systems has rapidly increased and become popular. Extensive survey studies show that they are increasingly being adopted in conversational AI, recommendation systems, and domain-specific question answer applications. [2]. This studies highlight that retrieval quality, chunking strategies, embedding representation and ranking mechanisms are critical factors affecting overall system performance.

2.1 Current Approaches and Methodologies

Semantic search is a fundamental component of modern RAG systems. Traditional sparse search approaches like TF-IDF and BM25 rely heavily on lexical overlap between queries and documents, resulting in limited ability to capture semantic similarity. To overcome this problem, DPR was introduced as a neural search framework that maps both queries and passages to a shared dense vector space.[3]. DPR significantly improves information retrieval accuracy by utilizing transformer-based encoders.

The effectiveness of dense search systems largely depends on high quality embedding

models. SBERT is one of most commonly used embedding architectures for semantic similarity tasks. [4]. Unlike traditional BERT models that require binary inference, SBERT generates fixed-length sentence embeddeds that allow for efficient similarity calculations using cosine distance or nearest neighbor search. This architecture makes it feasible from a computational standpoint.

Vector similarity search is another fundamental requirement in RAG. Since embedding-based access works on high dimensional vector representations, scalable indexing mechanisms are needed to support low latency access on large datasets. FAISS is one of the most widely used vector indexing libraries for this purpose. [5]. FAISS provides optimized nearest neighbor search algorithms and gpu acceleration, enabling billion scale similarity search in production environments.

Beyond static search pipelines, recent research has explored adaptive, high performance and iterative search mechanisms. RAG utilizes dynamic searching throughout the process, rather than searching documents only once at the beginning. [6]. This approach significantly reduces hallucination and enhance realism by continuously updating contextual accuracy throughout text creation process.

Hallucination reduction remains one of the major motivations behind retrieval augmentation. Conversational systems integrated with retrieval components generate more factual and reliable responses than purely generative dialogue models [7]. Retrieval mechanisms improve both response quality and reliability by basing generated outputs on verifiable sources.

Another important research direction involves domain customization in systems that enhance access to information. While basic RAG architectures operate effectively in open-domain datasets, deploying these systems to specific enterprise or business environments presents additional challenges, such as limited training data, various incompatibilities, restricted information access and permission requirements. Domain-specific techniques are essential to maintain information access and production performance in closed-domain applications.[8].

2.2 General Evaluation and Trends

A comprehensive review of the literature reveals several key elements.

1) There is a shift from purely generative architectures to hybrid knowledge acquisition and generation systems. While classic LLMs provide fluent and human-like responses, they often produce misleading or unverifyable information. Knowledge

acquisition supporting architectures address this limitation by basing the generated responses on the resources available within the system.

2) Recent research increasingly focuses on improving search quality through better embedding representations, optimized chunking strategies, hybrid search pipelines, and reordering mechanisms. Survey studies show that search performance directly impacts production quality and success in RAG systems.[2].

3) Scalability and efficiency become critical in real-world applications. The adoption of vector databases and efficient indexing libraries like FAISS demonstrates the increasing importance of low latency and fast data access in large scale systems. [5]. Modern enterprise tools require architectures that can process ever-growing datasets, produce efficient and satisfactory outputs and maintain fast response times.

4) Another emerging trend is the development of adaptive retrieval pipelines. Instead of rely on a single retrieval stage, modern systems increasingly incorporate iterative retrieval, query reformulation, and retrieval feedback mechanisms to improve contextual relevance during generation [6].

5) RAG applications increasing attention in academia and industry. Enterprise environments require systems ensure data confidentiality, controlled information access, explainability and source verification. These requirements present additional challenges compared to other systems.[8].

2.3 Conclusion

In conclusion, the literature demonstrates that the RAG approach provides a robust framework for building advanced question answering systems and is a more sensible structure in this regard compared to classical fine-tuning methods. Dense retrieval approaches like DPR improve semantic matching capabilities, while embedding models like Sentence-BERT enable efficient representation learning for similarity-based retrieval. Additionally, vector indexing technologies like FAISS support scalable real time retrieval on large datasets.

The studies reviewed also show that improving information acquisition processes significantly reduces hallucinations and increases consistency and reliability in the responses generated. On the other hand, adaptive information acquisition techniques and domain-specific optimization strategies, parameter testing, are becoming increasingly important for enterprise level applications.

Nevertheless, several challenges remain to be addressed. These include:

- Ensuring secure and controlled access to enterprise data,
- Adapting RAG systems to different domains,
- Increasing the accuracy and precision of the data retrieval process,
- Ensuring that the generated responses are explainable and verifiable with sources,
- Continuously evaluating and improving system outputs

3
System Analysis and Feasibility

3.1 System Analysis

The system analysis phase aims to define and determine the core components, functions, and requirements of the proposed system in order to identify the most suitable solution. In this context, the project aims to design a structure that works with internal data and complies with accurate, relevant documentation.

The system is designed to process unstructured institutional documents, such as pdfs and procedural documents, and make them easily searchable. To do this, the system first extracts the text, cleans up irregular formatting, breaks it down into smaller, manageable chunks and converts it into a mathematical format that artificial intelligence can understand. Once the data is neatly organized, our search engine uses it to quickly find and extract the exact information the user is looking for. These chunks are then evaluated based on cosine similarity and the most similar chunks fed into the LLM for answer. This structure will be achieved by uploading all pdfs to the system in chunks and generating output in the models prompt by only referring to the relevant documents.

On the other hand, the system is expected to minimize incorrect or misleading outputs and provide consistent and reliable responses, with alerts adjusted accordingly. In addition, the model is required to specify the source for each piece of information it provides, and finally, list all sources used.

The main functional components of our system can be summarized follows:

- Document processing and data preparation
- Text segmentation and chunking
- Transformation of text into vector representations => embeddings
- Storage and indexing of embeddings in a vector database => chromaDB

- Initial retrieval of relevant information based on user queries
- Cross-encoder reranking and threshold filtering for better context precision
- Natural language response generation guided by prompting strategies
- Model's answers along with source references

Beyond ensuring the system worked, a reliable method was needed to measure its actual performance. To evaluate success, the RAGAS method, a popular method in the literature that automatically scores the system based on three key criteria, was used. However, these scores are not automatically calculated by LLMs but rather based on various metrics and formulas.

- First, it was checked whether the AI was completely truthful based on the documents and whether it was fabricating information -> Faithfulness.
- Second, it was measured whether the search engine retrieved the correct documents without adding irrelevant noise -> Answer Relevancy.
- Third, it was evaluated whether the final response directly answered what the user was actually asking -> Context Precision.

Meeting these three criteria is the final test proving that the system is doing its job accurately and reliably.

Finally, the project will be completed with a simplified, classic chatbot interface that named Charm-AI.

3.2 Feasibility

3.2.1 Technical Feasibility

The technical feasibility phase aims to analyze the hardware, software, and platform selections required for the project to achieve its main objective.

The system involves processing unstructured textual data, generating vector representations, and using LLM to produce the desired responses. Therefore, the feasibility analysis focuses on evaluating suitable software tools and hardware infrastructure that can efficiently support these processes.

The biggest technical challenge of the project is the time and hardware load created by dividing the 130GB database of 8000+ PDFs into chunks. To reduce both time

and hardware load, the project utilized Google Colab and Kaggle platforms with higher-performance processors instead of using local computers. Furthermore, the limitation on data loading size was addressed by loading the data in chunks and adding them together.

Another challenge was that the documents could contain different languages, requiring a suitable solution. For this, the BAAI/bge-m3 model, which is suitable for each language and provides a high success rate, was integrated and resolved.

3.2.1.1 Software Feasibility

The project's software infrastructure was selected based on machine learning standards and tools that offer the highest performance for LLM. Python was chosen as the main development language due to its extensive library support, rapid prototyping capabilities, and simple structure.

The system includes components for document processing, text cleaning, embedding and retrieval operations. The pdfplumber library was used for this purpose. Additionally, a vector database is required to store and efficiently query high-resolution embedded data. ChromaDB, one of the most popular and well liked database was chosen. Furthermore, since we added a reranker structure later in project, robust and successful BAAI/bge-reranker-v2-m3 model was selected. For the LLM, three different models were tested and their results compared. Experiments were also conducted by making changes to various metrics.

Of course, different alternatives exist for each software component; however, the chosen solutions prioritize ease of integration, security, performance efficiency, and flexibility. These factors ensure the scalability, sustainability and consistency of the system.

3.2.1.2 Hardware Feasibility

Hardware feasibility analysis determines the processing power required to ensure the system runs smoothly and efficiently. The heaviest computational loads naturally stem from processing large raw documents and breaking them down into thousands of smaller pieces, converting these pieces into mathematical vectors, performing rapid similarity searches and finally running the language model to generate responses. This is because our current dataset contains thousands of pdfs and over 100 GB of data in total. The solution to this challenge, as mentioned in the technical feasibility analysis, to perform these operations by leasing high-powered processors from various

platforms.

Sufficient processing power, memory capacity and storage space are required for the system to run. Processes such as document processing, chunking and embedding consume significant resources as data size increases. The minimum system requirements used are given in Table 3.1.

Table 3.1 Minimum System Requirements

Component	**Minimum Requirement**
CPU	Multi-core processor(4 or more)
RAM	Minimum 8 GB RAM (16 recommended)
Storage	Minimum 150 GB available SSD storage
GPU	Cloud GPU or local GPU support for faster embedding generation
Operating System	Windows, Linux, or macOS
Python Environment	Python 3.10 or newer

These requirements are considered sufficient for the system to perform its basic functions. In cases of larger document sizes or higher transaction volumes, both disk space and additional RAM capacity, along with GPU support, can significantly reduce processing time. This configuration allows the system to handle the current volume of data while remaining adaptable to future data increases.

3.2.2 Legal Feasibility

The legal viability of the project focuses on whether the development process complies with applicable data usage policies, software licenses, and proprietary requirements. Since the system relies on external documentation, open-source tools, and language model services, the following aspects are considered:

- **Dataset Usage:** This project utilizes reports and documents gathered from publicly available sources, including international organizations such as UNESCO and other official sources. These documents will be used solely for academic research and systems development purposes and no alterations will be made to them.
- **Libraries and Tools:** All software libraries, frameworks and vector database components used in the system have been selected taking into account license terms and permitted usage.

- **Language Model Usage:** The selected LLM services like Llama,Qwen and API services like Groq,Openai are used in accordance with their usage policies and licensing terms.
- **Commercial Restrictions:** The current project is designed for research and evaluation purposes and does not involve the commercial redistribution of data.
- **Name Usage:** Since there is no other chatbot named Charm-AI on the internet, there are no issues with naming rights.

In general, the project is legally complete as long as documentation usage policies, software licenses and attribution requirements are followed throughout development and deployment.

3.2.3 Economic Feasibility

Since this project is a research study rather than a commercial product, its economic feasibility is based on obtaining maximum academic and strategic value with minimum cost, rather than income and expense balance.

The main cost considerations are summarized below:

- **Hardware and Infrastructure Costs:** Processing large, thousands of documents and creating embedded data requires computing resources such as CPU, RAM, storage and optionally GPU-based acceleration. Additional cloud resources can be used to reduce processing time during indexing and testing phases. Initially, Kaggle offers a limited free trial period; if this is insufficient, a monthly Google Colab subscription (165 TL) can be considered.
- **Software and Service Costs:** All libraries used in the project are under open-source licenses. Models such as Llama are used free of charge within certain limits by creating API keys under community licenses. If these limits are insufficient, it may be necessary to top up your credit balance for use. The datasets are publicly available. No costs were incurred for these products.
- **Development and Maintenance Costs:** The project was carried out by an undergraduate student within one academic semester. Therefore, there are no labor costs.

3.2.4 Workforce and Time Planning

The project timeline is planned by dividing it into multiple phases; including data preparation, system development, model integration, evaluation testing and interface design. Each phase requires technical knowledge in areas such as natural language processing, machine learning and software development.

The estimated duration for each task is projected on a Gantt chart based on its complexity and application requirements. The development process is managed by a single student with knowledge and interest in artificial intelligence and software engineering, and is expected to be completed in approximately 3 months.

Gantt Chart showing the project timeline in figure 3.1

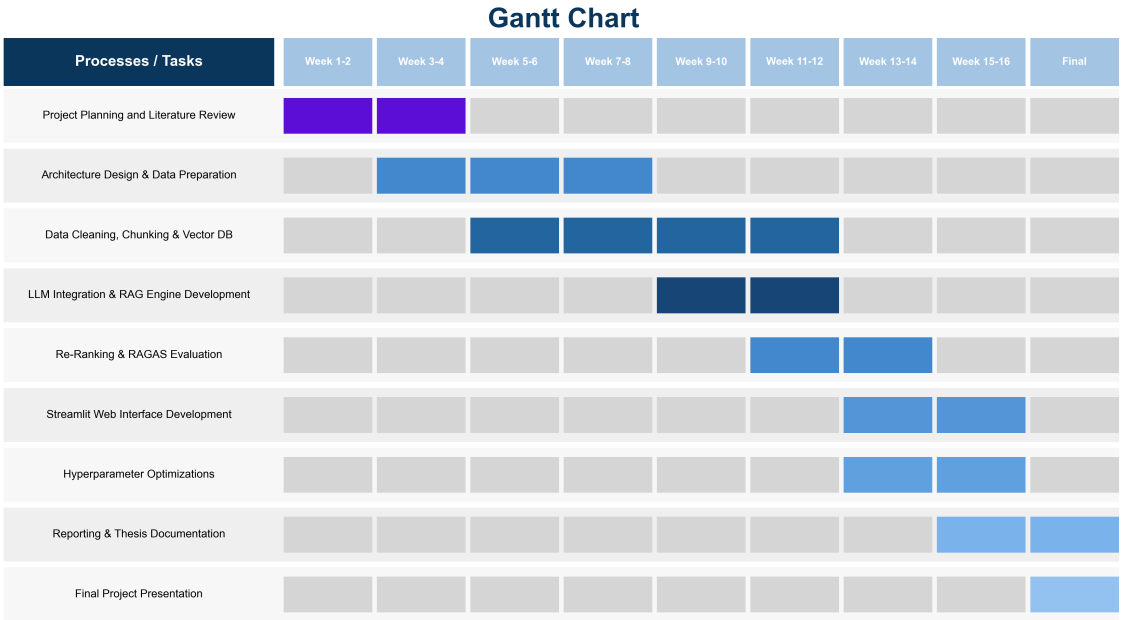

Figure 3.1 Gantt Chart

4
System Design

This section presents the general architecture and operational steps of an organization-specific intelligent question-answering system developed using the RAG approach. The project aims to generate more accurate and contextually appropriate answers by utilizing relevant information available in the database, rather than solely relying on the internal knowledge of the large language model.

Unlike the classical generative language model approach, the system is designed to first find relevant document chunks and then generate an answer using these chunks, rather than directly answering the users question. To this end; data processing, chunking, embedding generation, vector indexing, semantic search, reranking, and LLM-based answer generation steps are combined. Finally, a chatbot interface called Charm-AI was developed, supported by a visually simple and elegant design.

A block diagram is provided showing the basic processing steps in the system, starting from the dataset phase and extending to the final evaluation metrics in figure 4.1.

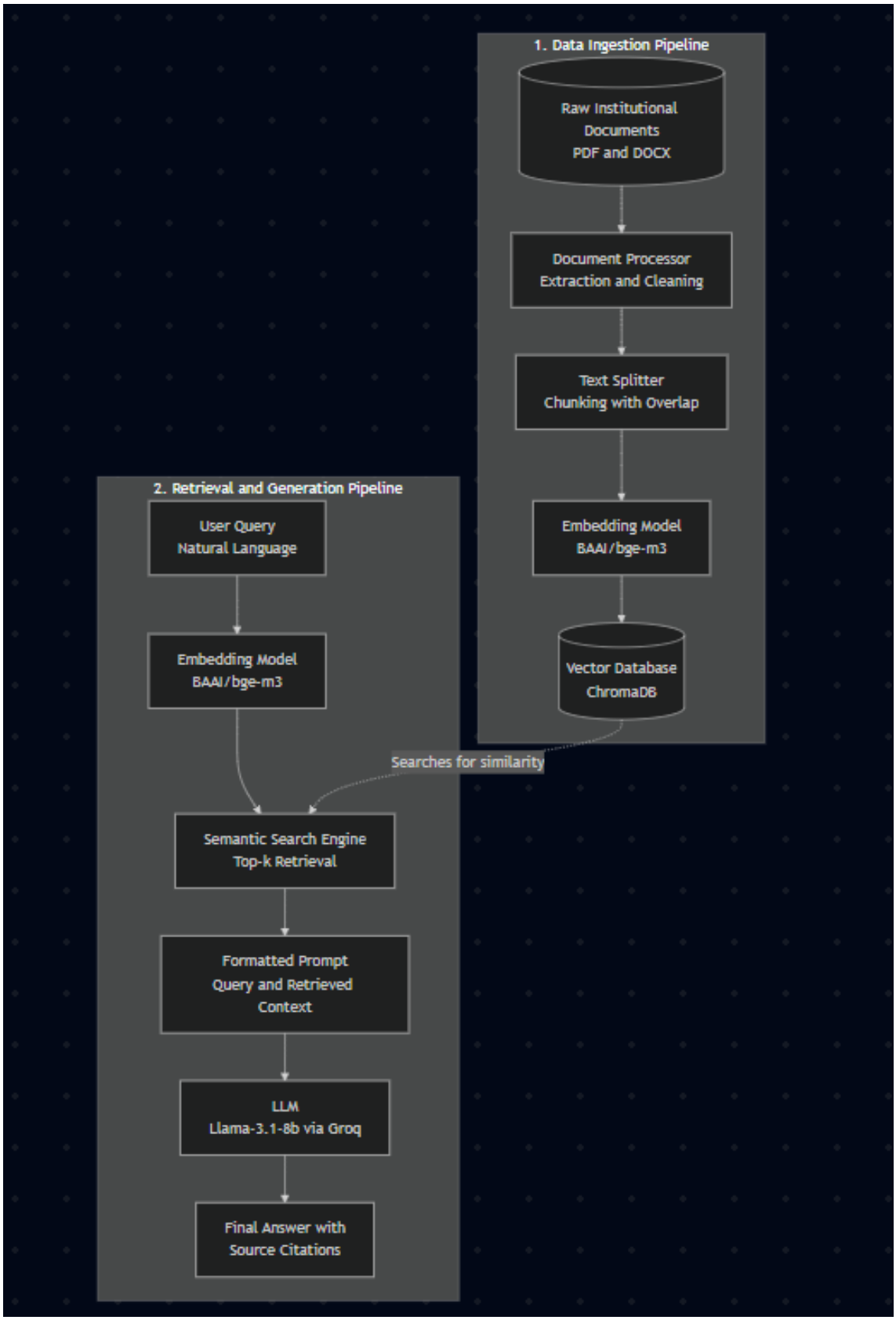

Figure 4.1 Block Diagram of the RAG-Based QA System

The sequence of operations within the system is as follows:

- **Data Ingestion and Preprocessing:** The relevant pdf files are retrieved and the textual content is extracted using pdfplumber. The extracted text is then cleaned to remove corrupted characters and unnecessary spaces.
- **Text Chunking:** The cleaned text is broken down into smaller, overlapping fragments to fit within the entry boundaries of the embedding pattern while preserving the semantic context.
- **Vectorization and Storage:** Each piece of text is processed by the embedding model to create vectors. These vectors are then indexed and stored in a chromadb along with metadata such as the source file name and page number.
- **Semantic Search:** When a user submits a query, it is vectorized using the same embedding model. The system calculates the similarity between the query vector and the document vectors using the cosine similarity method and selects the k segments with the highest score.
- **Cross-Encoder Reranking and Filtering:** The initial candidate data chunks are processed by the cross-encoder model along with the user's question, relevance logit scores are calculated, and k(40) of the highest scoring chunks are retrieved first. The data chunks are reordered, filtered by passing through a similarity threshold to remove noise, and finally k(15) of the most relevant data chunks are selected, and the model uses these chunks.
- **LLM Generation:** The resulting data chunks and user question are converted into a prompt. This prompt is answered by the LLM, which generates a contextual and verifiable response based solely on the provided documentation.

4.1 Material and Dataset

The core material for this project includes a comprehensive collection of institutional documents, advisory board assessments, conservation status reports, and global risk assessment studies published by international organizations such as UNESCO. This dataset has been meticulously prepared by experts in the field and by the person who will use the project after its completion. The raw dataset contains over 8.000 documents with a total size of approximately 133 GB and consequently; when divided into chunks, it contains enormous numbers and sizes of chunks.

Because the documents contain various formatting inconsistencies, images, and non-textual content, they cannot be used in their original form. Instead, they undergo preprocessing and an extraction of unnecessary text. The extraction process

removes corrupted characters, unnecessary spaces and formatting errors, separating only the semantically meaningfull and useful textual content from each document. This reduction step significantly compresses the data size while preserving all the informational content of the dataset.

Because processing the entire document simultaneously is highly inefficient and exceeds the limits of generative models and because the method defined in the project is to chunk the text and use the k most relevant chunks, cleaned text is systematically divided into smaller chunks. A recursive technique, popular in the Iiterature, is applied to divide the text into 1000 character chunks with a 150 character overlap. This overlap strategy aims to prevent semantic fragmentation by ensuring that sentences cut at the boundary of one chunk retain their meaning in the next. We could have chosen a larger chunk size, but this was specifically preferred because relatively small chunks produce sharper, more semantically focused vector embeddings; this increases accuracy and the strength of the project, even at the cost of requiring a higher k value to achieve full contextual coverage.

In terms of data structures and storage, pdf documents are stored in a local directory system, while processed chunks are converted into vectors. These vectors, along with relevant metadata such as source filenames and page numbers are stored in chromaDB which is highly popular and preferred vector database. The resulting vector index occupies significantly less storage space than the original 133 GB raw dataset, enabling fast and similarity-based access during system operation.

4.2 Method

The fundamental methodology of this project is based on RAG which is relatively new architecture that combines traditional knowledge retrieval mechanisms with generative LLMs[1]. Unlike independent generative models that rely on pretrained parametric memory, RAG reduces the commonly observed problem of hallucinations by basing the generation process on external, verifiable information sources obtained in real time. Furthermore, it requires no training and accomplishes this using data already present in the system.[7].

The BAAI/bge-m3 model is used to create the embedding vectors. This advanced model transforms text fragments into dense, continuous vectors of 1024 dimensions. Dense embedding models have proven highly effective in capturing the deep semantic meaning of texts. The model we have chosen perfectly suits the project's requirements with its robust structure and multilingual support. Therefore, its slightly heavier structure compared to simpler models has been overlooked. [3].

The initial retrieval phase is managed by ChromaDB, an open-source vector database optimized for AI workloads. ChromaDB performs efficient similarity search using the HNSW algorithm [9]. HNSW is an ANN search algorithm that constructs a multilayer graph to achieve logarithmic time complexity during vector retrieval. When a user asks a question, this question is embedded using the BAAI/bge-m3 model, then ChromaDB calculates the cosine similarity between the query vector and the stored document vectors within the HNSW graph.

To further improve the quality of the context obtained, a two stage retrieval structure was used with the Cross-Encoder Reranker.For this rerank mechanism, the BAAI/bge-reranker-v2-m3, another powerful and popular model was chosen. Although the initial dual encoder model was extremely fast, it lacked the necessary deep structure and did not fully deliver the success we expected. Furthermore, since our data contained numerous documents from previous years on the same topic, we needed to fetch more chunks than anticipated. Therefore, the reranker was used, and its benefits were observed. The reranker processes the initial candidate chunks by feeding the question and each document together into the neural network, generating highly accurate relevance scores. The system then ranks these candidates and selects the k best chunks to be passed to the generation phase. Moreover, it did not prolong the expected response time when the question was asked as frequently as anticipated, thus increasing success.

Before reranking, a thresholding mechanism was also integrated into the pipeline. Even the highest-ranked documents might be fundamentally irrelevant if the database doesn't contain information relevant to the user's specific query. Therefore, I tried setting a threshold and conducting experiments. Both this structure, the rerank structure, and structures without either were analyzed in detail, and their performance was measured. The results and analyses are examined in more detail in the following sections.

For the production phase, the Llama 3.1 8B model is integrated into the processing pipeline via the Groq API, enabling ultra-fast inference through dedicated LPUs. Advanced language models like Llama are largely instruction-oriented, using RLHF and SFT to optimize contextual reasoning and instruction following. Three different models were tested in the project, including models with larger parameters, but the Llama 3.1 8B model proved more than adequate. It performed well in tasks that did not require training and involved processing pre-trained and readily available data. The tests also showed that choosing a model with larger parameters would be a significant time disadvantage. [10].

Subsequently, various prompt structures were tested. We tried a base prompt for our initial tests, followed by another prompt with stricter rules and finally a step-by-step prompt called the Chain of Thought. Measurements showed that the Chain of Thought prompt achieved the highest scores and was therefore designated as our main prompt for the project. Our prompt, which included strict rules, performed very poorly, falling far short even of the base prompt. Because of these strict rules, the system even responded to questions containing existing information with "This information is not available" to avoid taking any risks.

Up to this point, there was no problem, but we were evaluating the questions one by one, from a human perspective. However, this method was neither objective nor consistent with individual perspectives, and it was also quite time-consuming and tiring. Therefore, it was determined that a different evaluation metric was needed. The necessary research was conducted, and the RAGAS metric, which is popular, on the rise, and widely used in the literature, was chosen. This metric has various metrics and scoring formulas, and the application of these formulas and evaluations is left to a large language model. However, it does not rely on random evaluation and produces consistent results. When the answers given by our system and the RAGAS[11] scores were examined by relevant experts, positive comments were received. The system is evaluated targeting 3 different metrics:

Context Precision => evaluating the effectiveness of the reranker and threshold mechanisms,

Faithfulness => ensuring the LLM does not hallucinate beyond the retrieved context

Answer Relevancy => measuring how directly the generated response addresses the query.

In addition, the same questions were asked to models such as Gemini and ChatGPT, as well as to the model we used in our project, and various comparisons were made, objectively comparing the good and bad aspects. This will be examined in detail during the performance evaluation phase.

In the final stage, the goal was to create simple and easy-to-use software, similar to improved programs and classic chatbot developments. The chatbot was named Charm-AI, inspired by the structure of the RAG system.

5
Experimental Results

In this study, various experiments were conducted to evaluate the performance of the proposed RAG system. The experiments aimed to assess the system's accuracy, resistance to hallucinations, and potential improvements. First, a comparison was made between the Gemini model and the system in our project. The same questions were asked to both models to determine the advantages and disadvantages of our model. After identifying our shortcomings, we focused on the parameters that needed to be changed, and various experiments were conducted under constant and identical conditions using three different models. The aim was to measure the impact of small and large parameter models on success, and the impact of different types of models on success. The selected models were: Llama-3.1-8B, Llama-3.3-70B, and Qwen-32B. Furthermore, to answer the question of how the number of selected documents affects success, the k=(3,5,8,10,15) metric was tested under the same conditions. Finally, considering the negative impact on the answer when using a large k, especially when the most relevant chunks are ineffective, a threshold was decided upon. Experiments were conducted by applying threshold values(0.50, 0.55, 0.60) to the cosine similarity scores under equal conditions. Although an increase was observed, it was not a large jump, so Cross-Encoder Reordering was tried. Finally, 3 different prompt types were tested, and the highest scores were combined to find the combination with the highest score.

5.1 Qualitative Comparison: Charm-AI vs. General Purpose LLM

A direct comparative analysis was conducted between the RAG system and the most advanced LLM (Google Gemini). Both systems were asked the same questions to identify their differences, advantages and disadvantages.

Figure 5.1 shows the response of a general-purpose LLM to the question, "What is the importance of cross-border cooperation following war or major disasters?". While the response is fluent and well-structured, it is entirely based on the model's memory and

contains general information, lacking sufficient detailed technical information.

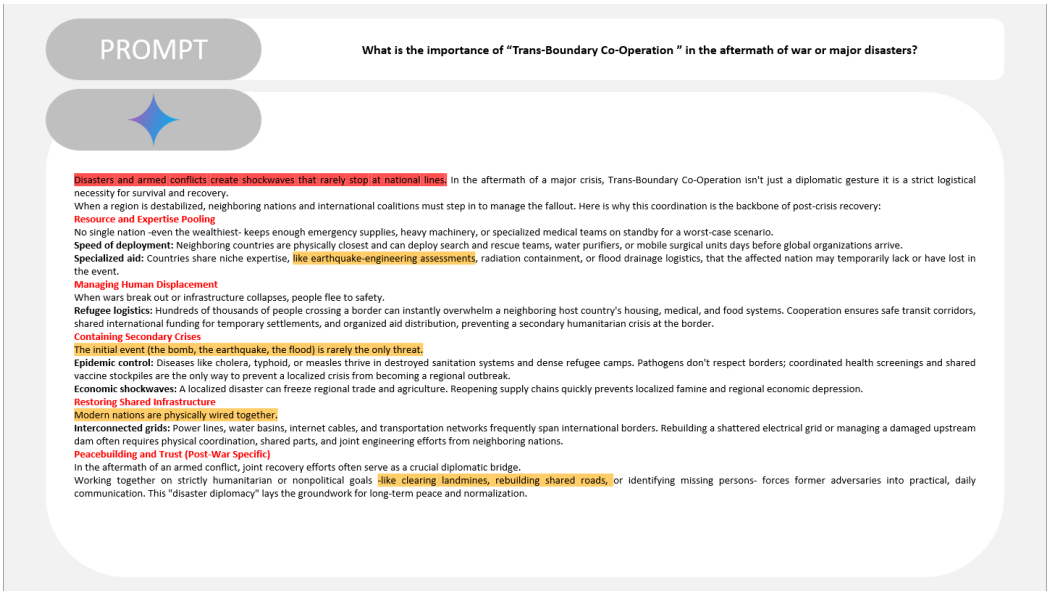

Figure 5.1 Response generated by Google Gemini.

In contrast, Figure 5.2 shows the response generated by the proposed RAG system for the same question. The system retrieves information just from the corporate document database and extracts detailed, expert-reviewable information from documents in seconds by specifying sources in the form of document names and pages.

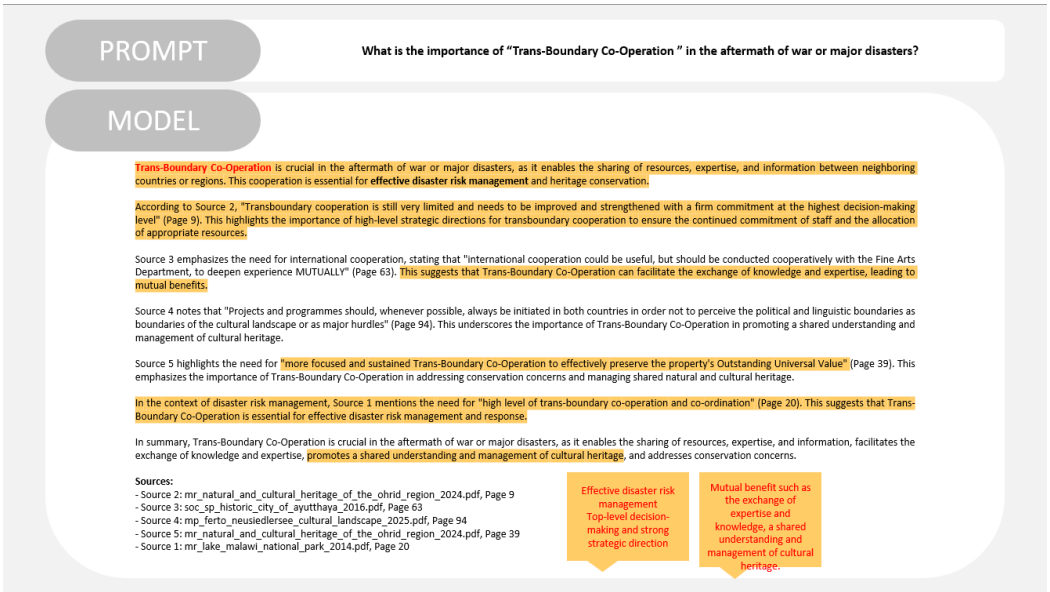

Figure 5.2 Response generated by Charm-AI

An objective comparison between the two systems is summarized in Figure 5.3. The RAG system demonstrates the characteristic of providing consistent and reliable,

domain-specific answers, direct responses that also specify and illustrate source references, and a strict refusal to answer questions outside its scope. In contrast, while the general-purpose LLM is more flexible in tone and scope, it risks providing unverifiable or speculative content, especially in specialized technical fields. It also misses critical domain-specific information and offers more general information.

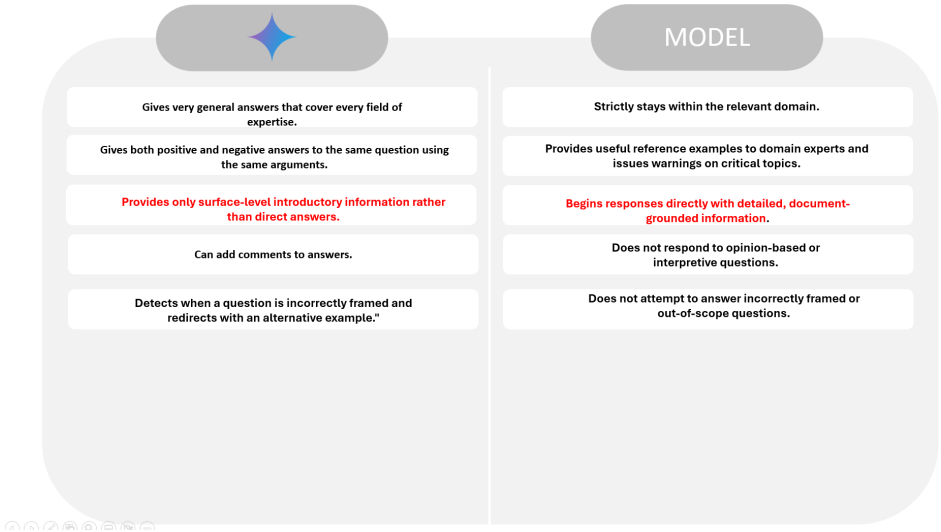

Figure 5.3 Comparison between the Gemini and Charm-AI.

Figure 5.4 illustrates two critical hallucination resistance behaviors of the proposed system in more detail. In the first example, a question based on opinion regarding the future delisting of Syrian World Heritage sites is deliberately posed. Instead of generating an unsupported prediction, the system correctly identifies that the relevant information is not present in the documents and refuses to respond. In the second example, a factually fabricated question referencing a non-existent document (a 2015 conservation report for a bridge built by Sinan in Paris) is submitted to the system. The system successfully avoids generating a seemingly logical but incorrect answer, instead transparently returning the response "This information is not available in the provided documents." This proves that by strictly filtering search results and forcing the AI model to stick only to the provided documents, the system successfully stops generating unproven answers.

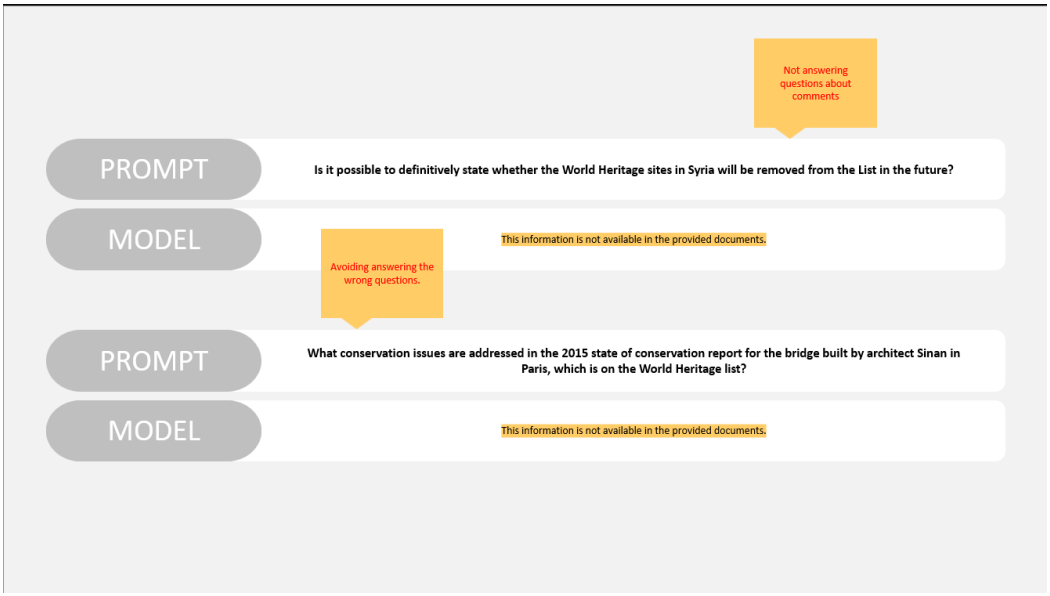

Figure 5.4 Resistance accuracy to hallucinations and incorrect questions.

These observations are consistent with the results obtained from the RAGAS evaluation and demonstrate that the RAG-based architecture achieves its core objective: To provide accurate, verifiable, and domain-specific responses that general-purpose models cannot reliably deliver.

5.2 Performance Analysis

5.2.1 Evaluation Metrics

To ensure an objective and reliable assessment, the system was evaluated using the automated RAGAS framework and manual expert scoring. The main criteria used in this study are as follows:

- **Faithfulness:** Measures how consistent the generated response is with the chunks the model sees, and whether the model is hallucinating. In other words, it measures how much the LLM relies solely on the provided documentation and how unaffected it is by external information.
- **Answer Relevancy:** Focuses on assessing how pertinent the generated answer is to the given prompt. A lower score is assigned to answers that are incomplete or contain redundant information and higher scores indicate better relevancy. This metric is computed using the question, the context and the answer.
- **Context Precision:** Measures whether the relevant parts were actually retrieved in the correct order. This metric is particularly important for seeing the effect of

reordering and thresholding mechanisms.

- **Weighted RAG Score (WRS):** The Weighted RAG Score was formulated to provide a single, unified metric for comparison. Given the critical importance of hallucination resistance in the project, *Faithfulness* was assigned the highest weight (1.0). *Context Precision* was prioritized second (0.8) as the foundation of retrieval quality, followed by *Answer Relevancy* (0.6). The WRS is calculated using the following normalized formula:

$$WRS = \frac{(\text{Faithfulness} \times 1.0) + (\text{Context Precision} \times 0.8) + (\text{Answer Relevancy} \times 0.6)}{2.4}$$

- **Blind Human Evaluation:** Prior to using the RAGAS metric, a human-led blind evaluation process was conducted. In this process, individuals familiar with the architectural structures and the content of the dataset evaluated the model's responses. Thanks to the feedback received in the initial and mid stages, the project progressed to the next phase. However, this evaluation method, which involved examining individual sources, was not preferred in later stages of the project because it was inefficient in terms of time and lacked objectivity.

5.2.2 Summary of Results

Data obtained from the automated RAGAS assessment are presented below. The tables and figures below include Faithfulness, Answer Relevance, Context Precision and Weighted Score for each parameter combination.

5.2.2.1 Comparison of Language Models (Baseline Setup)

In the initial stage of the experiments, no cross-encoder or threshold was applied, and the resulting chunk value, k, was set to 5. Furthermore, the following prompt, which serves as the base prompt, was used throughout the process leading up to the prompt comparison:

> *"You are an expert assistant specialized in architecture, heritage conservation, and disaster risk management. Answer questions ONLY based on the provided context documents. If the answer is not found in the context, state that the information is not available in the provided documents. Always cite your sources (document name and page number) at the end of your answer."*

This prompt is intentionally designed to remain context-dependent, implement

mandatory resource attribution, and thus eliminate reliance on the model's parametric memory.

Table 5.1 shows the RAGAS evaluation scores of the three language models tested under the same framework.

Table 5.1 Performance comparison of different LLMs under baseline configuration ($K = 5$, No Threshold, No Reranker).

Language Model	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
Llama-3.1-8B	0.826	0.801	0.955	**0.863**
Llama-3.3-70B	0.830	0.801	0.962	**0.867**
Qwen-32B	0.816	0.825	0.941	**0.860**

Looking at the results in Table 5.1, all three models exhibited very similar performance in the basic RAG configuration, with Weighted scores ranging from 0.860 to 0.867. However, even among models with significant parameter differences, the difference was no more than 0.007. In particular, the massive Llama-3.3-70B model, with almost nine times more parameters, showed only a +0.004 increase compared to the Llama-3.1-8B model with much smaller parameters. Similarly, the Qwen-32B model also failed to meet expectations and even performed worse than the Llama-3.1-8B model, achieving a weighted score of 0.86. The conclusion that can be drawn from these results is that, on the RAG side, the dependence on the provided context significantly reduces the need for the model's own parametric memory in larger models, and therefore, high-parameter models cannot improve performance. Therefore, disregarding this very small performance increase, the faster, more limited, and more easily applicable Llama-3.1-8B model was preferred for the remainder of the project.

5.2.2.2 Impact of Retrieved Context Size (K Value)

Following model comparison, the effect of the resulting relevant chunk number was analyzed. Using the Llama-3.1-8B model, the effect on sensitivity and accuracy was observed by systematically varying the resulting chunk number k between 3, 5, 8, 10 and 15. No threshold value, no reranker. RAGAS evaluation results for these variations are presented in the Table. 5.2.

Table 5.2 Impact of varying k values on RAGAS metrics using Llama-3.1-8B.

K Value	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
$K = 3$	0.711	0.732	0.970	**0.803**
$K = 5$	0.826	0.807	0.955	**0.864**
$K = 8$	0.792	0.868	0.927	**0.856**
$K = 10$	0.831	0.864	0.919	**0.869**
$K = 15$	0.915	0.848	0.904	**0.895**

As seen in the table, the highest number we tried, k=15, created an unexpected difference, outpacing its closest competitor by approximately 3%. The most commonly used k values in the literature are 5 and 10. However, the optimal k value varies depending on factors such as the size and content of the dataset. The chunk size also plays a role. In our project, we opted for a slightly below-average chunk size to capture context and avoid including irrelevant topics in the same chunk. Even with a relatively small chunk size, we can conclude that choosing a larger k value improves performance. Another reason why a higher k value yields better results is that the existing dataset contains documents for the same topic every year (for example, documents for the Kathmandu Valley every year from 1980 to 2026).

Furthermore, since increasing the value of k increases success, experiments were conducted under the same conditions with k = 20, following the logic of increasing k until the success percentage decreases. However, it yielded worse results compared to k = 15, with a weighted score of 88.2. This value was 89.6 for k = 15. Therefore, it can be said that the best results were obtained with k = 15 at this stage.

5.2.2.3 Impact of Retrieval Score Threshold

A dynamic Access Point Threshold was implemented to improve the system's resilience to noisy or irrelevant contexts. In this experiment, the Llama-3.1-8B model was used with the k parameter fixed at k=8 and k=10. For each configuration, three different cosine similarity thresholds (0.50, 0.55, and 0.60) were applied to observe their effects on filtering before generating low-quality document fragments. The effects on the outcome are documented in Table 5.3.

Table 5.3 Impact of Similarity Score Thresholds on RAGAS metrics for k=8 and k=10 using Llama-3.1-8B.

K Value	Threshold	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
$K = 8$	*0.00 (None)*	0.792	0.868	0.927	**0.856**
$K = 8$	0.50	0.797	0.868	0.932	**0.860**
$K = 8$	0.55	0.826	0.845	0.937	**0.868**
$K = 8$	0.60	0.712	0.748	0.887	**0.779**
$K = 10$	*0.00 (None)*	0.831	0.864	0.919	**0.869**
$K = 10$	0.50	0.825	0.900	0.921	**0.876**
$K = 10$	0.55	0.843	0.846	0.914	**0.867**
$K = 10$	0.60	0.777	0.771	0.891	**0.814**

First, looking at the thresholds for k=8, setting a threshold of 0.55 gives the best result. A threshold of 0.50 was also quite successful, with a difference of only 0.008. However, a threshold of 0.60 was a bit too high and resulted in approximately 9% lower results because it discarded relevant and potentially useful components.

Looking at the threshold scores for k=10, the best score is achieved with a threshold of 0.50. However, a threshold of 0.55 also performed well, falling only -0.009 behind the 0.50 threshold. Based on these results, we needed to apply either a 0.50 or 0.55 threshold. Knowing that larger k values would be used in the project, and since it was observed that the 0.55 threshold for k=8 did not even surpass the score without any threshold applied, a threshold value of 0.50 was chosen based on k=10.

Furthermore, experiments for both k values showed that not applying any threshold resulted in worse outcomes, and while our method didn't achieve a huge leap in performance, it did improve it.

5.2.2.4 Impact of Cross-Encoder Reranker

To further enhance retrieval precision, a Two-Stage Retrieval architecture was implemented using the BAAI/bge-reranker-v2-m3 cross encoder model. In the baseline configurations evaluated previously, cosine similarity scores (ranging from 0.0 to 1.0) were used as thresholds to filter candidate chunks. However, cross-encoder models operate on a fundamentally different scoring mechanism: Instead of comparing independent vector representations, the re-sorter processes the query and each candidate document together via a neural network to generate a score. These scores are non-probabilistic and can take negative or positive values. A threshold can then be apply to filter out results with low relevance before passing the remaining context to the language model.

Here, it was again considered beneficial to apply a moderate threshold and the logit threshold of -2.0, which is also a moderate threshold, was chosen. -2.0 threshold used provides a permissive and effective filter: it reliably rejects clearly irrelevant documents while allowing relevant documents to pass, exhibiting a balanced and efficient structure.

This experiment evaluated the Llama-3.1-8B model by systematically varying the Initial K (30 and 40) against the Final K (10 and 15). The results are presented in Table 5.4.

Table 5.4 Impact of Cross-Encoder Reranking on RAGAS metrics using Llama-3.1-8B (Logit Threshold $= -2.0$).

Initial K	Final K	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
30	10	0.873	0.864	0.963	**0.901**
30	15	0.861	0.872	0.951	**0.894**
40	10	0.858	0.847	0.946	**0.885**
40	15	0.864	0.890	0.950	**0.899**

As demonstrated in Table 5.4, the integration of the cross encoder architecture yielded the highest overall performance across all conducted experiments. Specifically, the configuration utilize an initial k of 30 and a final k of 10 achieved a peak Weighted RAG Score of 90.1%, significantly outperforming the best single stage Naive RAG setup. The reranker successfully maximized Context Precision 96.3% by accurately surfacing the most semantically relevant documents, which in turn elevated Faithfulness to its peak 87.3%. Base on results, highest values, initial k=40, final k=15 and initial k=30, final k=10, were carried over to the next stage. Due to the very small difference (0.002) between them, neither was chosen to determine the best combination; both continued.

5.2.3 Impact of System Prompt Design

In the next stage, the system prompt provided to the model was also considered a tool affecting response quality. To analyze the impact of the prompt structure on RAGAS metrics, three different prompt strategies were analyzed under the same fixed configuration (k=10, no t.hold, no rerank) using the llama-3.1-8b model and the results compared.

Prompt 1 (Base Prompt) — Role-Based Strict Instruction:

> *"You are an expert assistant specialized in architecture, heritage conservation, and disaster risk management.*

Answer questions ONLY based on the provided context documents.
If the answer is not found in the context, say: 'This information is not available in the provided documents.'
Always cite your sources (document name and page number) at the end of your answer."

Prompt 2 — Rule-Based Extraction Instruction:

"Your task is to answer ONLY using the retrieved context documents.
Rules:
Use only information explicitly stated in the provided context.
Do NOT infer, speculate, interpret, generalize, or introduce unstated conclusions.
If the context is insufficient, incomplete, ambiguous, or the answer cannot be directly supported, respond exactly:
'This information is not available in the provided documents.'
Every factual statement must be supported by retrieved evidence.
Do not merge information from different sources unless the relationship is explicitly stated.
Preserve the original intent and wording of recommendations, findings, and conclusions whenever possible.
Prefer extraction over abstraction.
Cite supporting sources immediately after the relevant statement using:
(Document Name, Page Number)"

Prompt 3 — Chain-of-Thought & Hedged Soft-Fallback Instruction:

"Use the provided context documents to answer the user's question as thoroughly and accurately as possible.
Follow these steps:
1. Carefully read all context documents provided below.
2. Identify the parts most relevant to the question.
3. Synthesize a clear, well-structured answer using only the information found in the context.
4. If the context contains only partial information, provide what is available and briefly note what could not be found.
5. Do not add information from outside the provided context. If the context contains no relevant information at all, briefly state that the documents do not cover this topic.

6. Always end your response with a 'Sources:' section listing every document name and page number you used."

RAGAS results for these three prompting strategies are presented in Table 5.5.

Table 5.5 Impact of system prompt design on RAGAS metrics ($k = 10$, No Threshold, No Reranker, Llama-3.1-8B).

Prompt Strategy	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
Prompt 1 (Base)	0.831	0.864	0.919	**0.869**
Prompt 2 (Rule-Based)	0.614	0.517	0.896	**0.684**
Prompt 3 (CoT & Partial)	0.902	0.843	0.916	**0.892**

Looking at the results in Table 5.5, it is clear that the prompt structure directly affects success rate. We need to look Prompt 2 in particular. Observed significant performance decrease when using rule based prompt. The main reason for this is that the restrictive prompt is too rigid, preventing the LLM from flexibly interpreting and combining the available information. As a result, the model encountered more frequent error situations, and a significant decrease in system performance was observed. Furthermore, it often returned the answer "this information cannot be found" even to questions containing information already present in the prompt. However, another prompt we tried, known in the literature as chain-of-thought, provided approximately a 2.3% increase in performance. Considering we achieved results in the 85-90% range, this value, while seemingly small, is actually quite significant. To understand the reason for this increase, firstly, the base prompt was a simple prompt consisting of only 3-4 sentences. However, the CoT prompt is more detailed and directs the model to reason sequentially.

In conclusion, it was deemed appropriate to use the third prompt, the CoT prompt, in the project. However, due to the powerful impact of prompts on success, different and creative prompts should also be tried in the future.

5.3 Comparative Analysis

Building upon the performance scores presented in the previous chapter, this section will provide a detailed comparative evaluation and interpretation of the different configurations that yielded the best results.

The aim of this is to find the most suitable parameter combinations based on varying parameters and to proceed with the project accordingly.

5.3.1 The Balance Between Contextual Scope and Information Noise

Experimental data, contrary to the common assumption that higher k values inherently cause hallucinations, showed that increasing the Faithfulness value from 71.1% to 91.5% when k=3 to k=15 (Table 5.2). This refutes the aforementioned assumption. On the other hand, this increase also resulted in an increase in the answer relevancy metric, but a decrease in the context precision value. This decrease is expected and negligible, as this value measures whether the chunks are brought in the correct order. Evaluating the ordering of 3 chunks among themselves is a completely different task compared to evaluating it for 15 chunks. Nevertheless, the fact that the score in this metric remains above 90% is actually highly valuable. Therefore, it should be seen as a positive rather than a decrease. In conclusion, increasing the k value from 3 to 15 resulted in a 9.2% increase in the weighted score. However, when values greater than k = 15 were tested, the relevant scores began to decrease. Therefore, it can be concluded that the value giving the best results in its simplest form, without applying other metrics, was found for the k value.

Even if we retrieve the most relevant k chunks, there might be chunks that are not useful. Therefore, a threshold was applied. Looking at the comparisons in the table 5.3, setting a threshold of 0.50 is the most balanced and successful method. This threshold does not include chunks with similarity scores below 0.50, even if they are within the first k values, and these chunks are not passed on to the model. A threshold of 0.60, on the other hand, was too aggressive and negatively impacted the scores.

5.3.2 The Superiority of Deep Semantic Matching via Cross Encoder

Threshold reduces noise in single stage situations and has a positive effect on the structure, but its capabilities are still limited. In basic RAG structure, questions are placed and the similarity level is calculated using cosine similarity which is the geometric distance between two vectors. The cross-encoder used in the project is a two stage structure. First, a high k value is selected and a rerank is created among these k values, bringing the most relevant chunks to the final k value. Moreover, this structure uses a method based on deep semantic matching, not vector distance.

As shown in Table 5.4, implementing the reranking phase resulted in the most significant improvements across all configurations, particularly in the *Context Precision* field. Compared to the optimal single-phase setup (Table 5.3; $K = 10$, $T = 0.50$), the reranker context precision increased from 0.921 to over 0.960. This significant increase is due to the cross-encoder processing the query and candidate document together through layers of attention. This allows the neural network to assess the actual semantic relationship and contextual dependencies between the user's specific

question and the document's content, rather than relying solely on keyword or broad topic overlap.

As shown in Table 5.4, the rerank method achieved the highest scores by making a positive leap in success percentages. When compared to Table 5.3; k=10 t=0.50, the reranker context precision increased from 92.1% to over 96%. Moreover, the overall success rate which was approximately 87%, reached 90%. There was also no disturbing increase in the negative impact on response time, which is the fearing and most negative effect of this rerank structure. Four combinations were tested, with starting k values of 30 and 40 and ending k values of 10 and 15. The most successful and closest results were obtained with starting k values of 40 and ending k values of 15 and starting k values of 30 and ending k values of 10. These were then evaluated in final stage for performance measurement combinations.

5.3.3 The Influence of Cognitive Prompting Strategies

Prompt 1 (Role-Based Strict Instruction) By establishing a solid foundation, it achieved a high Response Relevance 86.4%. However, the dual feedback mechanism model often forced all or nothing responses. When faced with complex queries where information was fragmented, the model either rejected the question entirely or attempted to synthesize it to avoid rejection. This lack of cognitive flexibility led to occasional actual inconsistencies, limiting the faithfullness value to a moderate 83.1%.

Prompt 2 (Rule-Based Extraction) The strategy aimed to maximize factual purity by explicitly prohibiting inference, abstraction, and generalization. While designed to enhance Faithfulness, this strategy effectively cripled the LLM's natural language comprehension capabilities. By forcing the model to act solely as inference engine, it struggled to coherently integrate fragmented information from diverse sources. However, these rigid approaches prove ineffective, failing to generate responses even from readily available sources. The resulting responses were often disjointed and did not directly address the user's core objective, negatively impacting Answer Relevance (0.517) and overall response fluency.

Conversely, **Prompt 3 (Chain-of-Thought & Partial Synthesis)** emerged as a superior overall strategy by prioritizing factual accuracy over rigid structuring. By structuring the task into sequential cognitive steps, prompt 3 dramatically increased the faithfulness value to 90.2%. Although Answer Relevance (0.843 to 0.864) decreased, this small drop was far less significant than the jump in faithfulness, proving this balance to be extremely beneficial. The large gain in factual reliability more than compensated for the small drop in relevance, achieving the highest overall Weighted

Score (0.892). This confirms that a flexible, reasoning-based prompt is superior to rigid rules for complex field building.

5.3.4 Identification of Final Architecture

Throughout these numerous evaluations, different parameters of the RAG structure were tested and optimized. The analysis revealed that rerank setup with a starting k=30, ending k=10, and Logit Threshold Value =-2.0 yielded the highest results. Furthermore, the Prompt 3 thought chain and partial synthesis, enabling the LLM to reason sequentially and process partial information smoothly, significantly improved faithfulness and overall scores. It was also determined in previous tests that the llama3.1-8b model would be used.

The aim of this study was to identify the values that yielded the highest scores by combining the parameters that had achieved the highest results. However, the initial k=30, final k=10 rerank model, which had the highest scores, did not provide the expected increase when combined with other parameters that had achieved the strongest scores. Since this increase was not observed, it was necessary to try other combinations. The comparison below compares the combinations that yielded the 3 highest scores. This evaluations were conducted under equal conditions and within the framework of the llama3.1-8b model and CoT prompt (prompt 3), which is the basis for the ongoing project.

Table 5.6 Final comparative evaluation of top architectural configurations using Prompt 3 (Llama-3.1-8B).

Architecture Configuration	Faithfulness	Answer Relevancy	Context Precision	Weighted Score
Naive RAG ($K = 15$, Threshold=0.50)	0.917	0.877	0.909	**0.904**
Reranker (30 → 10, Logit=−2.0)	0.892	0.857	0.956	**0.905**
Reranker (40 → 15, Logit=−2.0)	0.923	0.915	0.952	**0.931**

The latest evaluations revealed that the most suitable structure for the relevant prompt and model is the rerank combination with initial k=40 and finish k=15. A large and dense chunk number at the beginning, followed by a purified but still contextually broad number, provided a perfect balance. Therefore, final version of project also incorporates this combination.

6
Conclusion and Discussion

This project successfully developed an advanced question answer system, using the RAG framework, that is specific to and faithful to relevant documentation in the fields of architectural design and cultural heritage preservation. To overcome the inherently misleading tendencies in LLMs, a large raw dataset of over 8,000 specific documents (approximately 133 GB) was parsed and compressed into a vector database that is popular and frequently used in studies, and also suitable for the project. This project achieved success by providing fast and accurate responses, despite the use of a robust model, particularly in the embedding model, and rerank tools that prolong response times. The project was completed with an extremely simple and user-friendly interface that works flawlessly. Metrics such as chat deletion, archiving, language support, and dark mode, which should be present in the interface, were fully added. Additionally, a database tab was added, allowing for the listing of all PDFs in the system, the addition of new PDFs, the creation and deletion of new chunks. User-friendly guidance, small warnings, and a user manual are also included. Voice chat support and sample questions are other features found in the interface. The chatbot was named Charm-AI, and the interface was built accordingly.

The extensive and rigorously conducted experimental phases produced clearly interpretable results demonstrating that the RAG structure and parameters directly influence the system's actual success and reliability. RAGAS evaluations revealed that the highest performance was achieved by combining a broad initial semantic search (Initial k=40) with a rerank phase (Finish k=15). This rerank structure provided the LLM with nearly perfect efficiency and desired outputs. More importantly, when combined with a thought chain prompt that allows the model to reason sequentially, the system achieved a Weighted RAG Score of 0.931, a feat not previously achieved in tests. In these tests, which measured the answers to 36 questions prepared by experts in the relevant field, all 3 metrics evaluated under this specific configuration exceeded 90% success rates, with Context Precision yielding a remarkable 95.2%, while Faithfulness, the most important metric in the project, achieved 92.3%, and Answer

relevancy achieved 91.5%. These comparative performance analyses have definitively proven that classical and simple information search approaches are insufficient for specialized datasets, demonstrating that our system performs exceptionally well.

Given the initial goals, the project fully achieved its objectives by creating a highly reliable and successful chatbot called Charm-AI, which identifies the source of architectural questions found in documentation, answers them computationally and time-efficiently, reliably, and successfully, and presents experimental results as proof.

For future researchers aiming to advance in this field, several exciting trends exist. The most critical advancement is the transition from a purely text-based RAG structure to a multi-mode RAG architecture. Implementing multi-mode embedding models will offer truly comprehensive assistance, enabling the system to process, retrieve, and evaluate visual architecture data alongside textual documents.

References

[1] P. Lewis *et al.*, *Retrieval-augmented generation for knowledge-intensive nlp tasks*, 2021. arXiv: 2005.11401 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2005.11401.

[2] Y. Gao *et al.*, *Retrieval-augmented generation for large language models: A survey*, 2024. arXiv: 2312.10997 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2312.10997.

[3] V. Karpukhin *et al.*, *Dense passage retrieval for open-domain question answering*, 2020. arXiv: 2004.04906 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2004.04906.

[4] N. Reimers and I. Gurevych, *Sentence-bert: Sentence embeddings using siamese bert-networks*, 2019. arXiv: 1908.10084 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1908.10084.

[5] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1702.08734.

[6] Z. Jiang *et al.*, *Active retrieval augmented generation*, 2023. arXiv: 2305.06983 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2305.06983.

[7] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, *Retrieval augmentation reduces hallucination in conversation*, 2021. arXiv: 2104.07567 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2104.07567.

[8] S. Siriwardhana, R. Weerasekera, E. Wen, T. Kaluarachchi, R. Rana, and S. Nanayakkara, *Improving the domain adaptation of retrieval augmented generation (rag) models for open domain question answering*, 2022. arXiv: 2210.02627 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2210.02627.

[9] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[10] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[11] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

Curriculum Vitae

FIRST MEMBER

Name-Surname: CEYHUN KIRCA
Birthdate and Place of Birth: 25.01.2003, Balıkesir
E-mail: ceyhun.kirca@std.yildiz.edu.tr
Phone: 0553 552 55 06
Practical Training: Kredi Kayıt Bürosu -DevOps
IBTECH A.Ş -Data Designer

Project System Informations

System and Software: Windows Operating System, Python
Required RAM: 8GB
Required Disk: 256GB